Cost-of-Living Data Platform

Cloud Data Pipeline, Search Taxonomy, and Analytics API

Tzz and Contributors

2026

Abstract

The cost-of-living data platform monitors Australian public discussion, media coverage, and official CPI indicators through a cloud data pipeline. It collects public records from Bluesky, Mastodon, and GDELT GKG archive files; normalises source-specific records into a unified raw schema; processes text into topics, sentiment labels, and quality flags; and serves chart-ready analytics through a FastAPI REST API. The public architecture keeps one API serving path, uses Fission for scheduled data collection, runs NLP processing through KEDA-scaled Kubernetes workers, and stores data in Elasticsearch with ILM-managed processed indices.

Contents

1 Product Scope

Cost-of-living pressure appears in household expenses such as rent, groceries, energy, fuel, healthcare, education, and debt. Official indicators provide stable monthly context, while public social and media records provide timely but noisy signals. The platform focuses on repeatable monitoring rather than causal inference:

- collect public source records on schedules;
- filter for Australian context and cost-of-living relevance;
- preserve source provenance and raw processing state;
- process records into topics, sentiment, and quality flags;
- expose REST endpoints for notebooks, dashboards, and operational checks;
- compare selected discussion patterns with official CPI observations.

The main analytical constraint is representativeness. Bluesky and Mastodon show public platform discussion, GDELT shows media coverage, and CPI shows official price movement. These sources should be compared, not collapsed into one sentiment score.

2 Data Sources and Taxonomy

Table 1: Source roles

Source	Role	Interpretation
Bluesky	Public social discussion	Short public posts with search-based collection and Australian context filtering.
Mastodon	Public social discussion	Public posts from multiple instances, useful as a second social signal.
GDELT GKG	Media coverage	Archive-based media records, treated as attention in news coverage rather than direct public sentiment.
ABS CPI	Official indicator	Monthly official comparison signal for selected consumer categories.

The taxonomy lives in `data/cost_of_living_keywords.csv`. It covers housing, groceries, energy, fuel, transport, eating out, healthcare, home goods, education, inflation, wages, and debt. Terms were selected from common Australian spending categories, CPI-aligned categories, and locally meaningful terms such as supermarket brands, public transport references, and healthcare language.

GDELT uses the public GKG master file list rather than rate-limited article search. Incremental harvesting and historical backfill share the same archive processor: read list metadata, download `.gkg.csv.zip` archives, verify checksums, extract CSV rows, filter for Australian cost-of-living records, and bulk index matching records.

3 Cloud Architecture

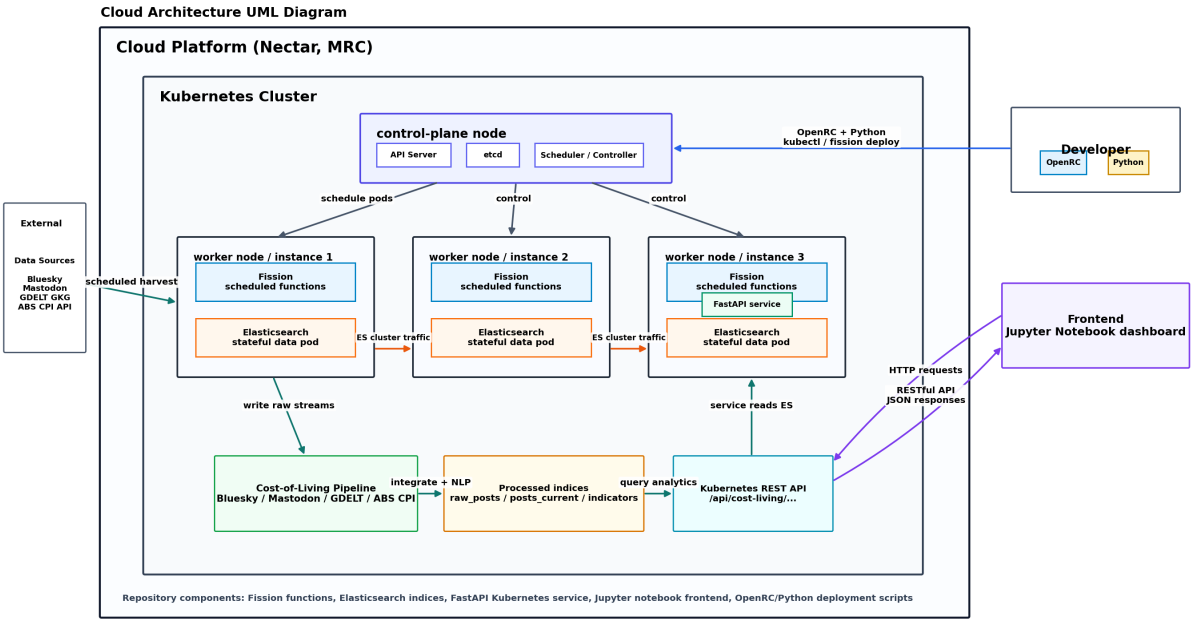

Figure 1: Pipeline architecture

The platform is organised around one public API path and several scheduled pipeline jobs. FastAPI serves analytics and operational endpoints under `/api/cost-living`. Fission runs scheduled harvesters, raw integration, and official indicator collection. Redis stores runtime locks, recent lifecycle events, short-lived API cache entries, and the NLP work queue. KEDA watches the Redis queue and scales NLP workers from zero. Elasticsearch stores raw records, processed records, CPI observations, and queryable processing status.

Table 2: Runtime components

Component	Responsibility
Fission harvesters	Collect Bluesky, Mastodon, GDELT, and CPI records on schedules.
Raw integrator	Convert platform streams into the unified raw schema and enqueue NLP work.
KEDA NLP worker	Consume Redis work items, claim pending raw records, clean text, classify topics, score sentiment, and write processed records.
FastAPI API	Provide analytics, health, cache, rate-limit, queue, and pipeline endpoints.
Redis	Coordinate locks, lifecycle events, shared API cache, active queue, and dead-letter queue.
Elasticsearch	Store raw streams, unified raw state, processed analytics, CPI indicators, and monthly rollups.

```
NAME                                                STATUS   ROLES           AGE   VERSION
comp90024-k3knzngdeexl-control-plane-mg4k9          Ready    control-plane   26d   v1.34.6
comp90024-k3knzngdeexl-default-worker-vw6nj-6wk7c   Ready    <none>          26d   v1.34.6
comp90024-k3knzngdeexl-default-worker-vw6nj-jnfz6   Ready    <none>          26d   v1.34.6
comp90024-k3knzngdeexl-default-worker-vw6nj-mhqbb   Ready    <none>          26d   v1.34.6
```

Figure 2: Kubernetes node layout

```
NAME                                        STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTE
SCLASS   AGE
elasticsearch-data-elasticsearch-es-default-0   Bound    pvc-3a1dea10-6da1-462a-8f3b-fce4fd341990   100Gi      RWO            perfretain     <unset>
         27d
elasticsearch-data-elasticsearch-es-default-1   Bound    pvc-cd0d5044-f6f4-4272-8f36-94d1fbbf515e   100Gi      RWO            perfretain     <unset>
         27d
```

Figure 3: Elasticsearch persistent volume claims

4 Elasticsearch and Queue Design

Table 3: Main indices and aliases

Index or alias	Writer	Reader
`cost_living_bluesky_raw_stream`	Bluesky harvester	Raw integrator
`cost_living_mastodon_raw_stream`	Mastodon harvesters	Raw integrator
`cost_living_gdelt_raw_stream`	GDELT harvester	Raw integrator
`cost_living_raw_posts`	Raw integrator	NLP worker, status API
`cost_living_processed_posts_write`	NLP worker	Elasticsearch rollover
`cost_living_processed_posts-*`	Write alias	API through read alias
`cost_living_posts_current`	Alias update	Analytics API
`cost_living_indicators`	CPI harvester	Comparison API
`cost_living_monthly_topic_metrics`	Rollup job	Optional API acceleration

Raw and processed records are separate. Raw records retain source provenance, source identifiers, original text, timestamps, and processing status. Processed records hold topic labels, sentiment, quality flags, and chart-ready fields. The API reads processed data through `cost_living_posts_current`, while the worker writes through `cost_living_processed_posts_write`. The processed backing indices use Elasticsearch ILM and rollover, so frontend routes remain stable while storage evolves.

Processing state is explicit. A raw record starts as `pending`, a worker atomically moves it to `processing`, and the final state becomes `processed`, `discarded`, or `error`. Stale `processing` records can be reclaimed after a configured timeout.

Redis handles runtime concerns that do not belong in Elasticsearch. The active NLP queue is:

```
cost_living_pipeline:queue:nlp
```

Messages have a bounded retry budget. Transient failures are requeued with attempt metadata and the latest error. Messages that exceed `NLP_QUEUE_MAX_ATTEMPTS`, cannot be decoded, or use an unsupported message kind are isolated in:

```
cost_living_pipeline:queue:nlp:dead-letter
```

The queue status endpoint reports both active queue depth and dead-letter depth, so worker failures are visible instead of silently looping.

5 API and Operations

The API is read-only for analytics and exposes operational endpoints for cloud checks:

Table 4: Endpoint groups

Endpoint group	Purpose
`/health`	API and Elasticsearch readiness.
`/overview`, `/topics`, `/sentiment`	High-level counts, topic distribution, and sentiment summaries.
`/trends/*`	Daily and monthly time-series views.
`/platforms/*`	Source comparison and plugin metadata.
`/cpi/*`	Official CPI comparison endpoints.
`/data-quality/*`	Quality flag and clean-vs-all comparisons.
`/pipeline/*`	Processing status, runtime events, and queue depths.
`/cache/status`	Redis or in-memory API cache diagnostics.
`/rate-limit/status`	API rate-limit configuration and backend.
`/metrics`	Prometheus request, latency, cache, queue, and rate-limit metrics.

Every request receives an `X-Request-ID`. Incoming IDs are preserved when supplied by clients, and generated IDs are attached to structured API logs. OpenTelemetry spans can be emitted through the configured exporter, and Prometheus metrics are exposed for scraping.

6 Validation Evidence

The current repository validation passes:

```
63 passed
Public release check passed for 122 release files.
```

Cloud smoke validation covered the public API prefix, queue status, rate-limit status, request-id propagation, Prometheus metrics, KEDA worker scaling, and processed-index writes through the rollover alias.

Table 5: Operational validation

Check	Result
API smoke test	25 endpoints passed.
Request tracing	`X-Request-ID` preserved in responses and logs.
Rate limiting	Redis-backed status endpoint available.
Prometheus metrics	Request, latency, cache, queue, and rate-limit metrics exposed.
KEDA worker	Scaled from zero, processed queue work, and scaled back down.
Elasticsearch ILM	Processed write and read aliases verified.
Public release check	Sensitive local markers and notebook outputs cleared.

7 Analytics Results

7.1 Processing Outcomes

Figure 4: Raw document processing outcomes

The processed corpus contains a large majority of usable records, with discarded records retained as visible processing outcomes instead of being deleted. This makes the pipeline auditable: filtering decisions can be checked through raw status fields and API diagnostics.

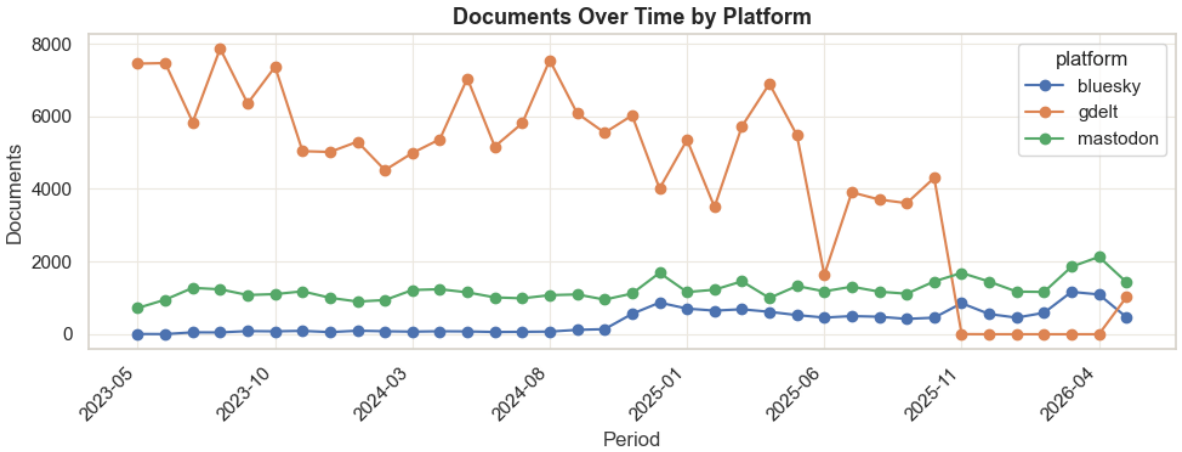

Figure 5: Documents over time by platform

Volume differs by source. Social sources are useful for personal and topical discussion, while GDELT is better interpreted as media attention. Keeping source groups explicit avoids mixing signals with different meanings.

7.2 Sentiment and Topic Patterns

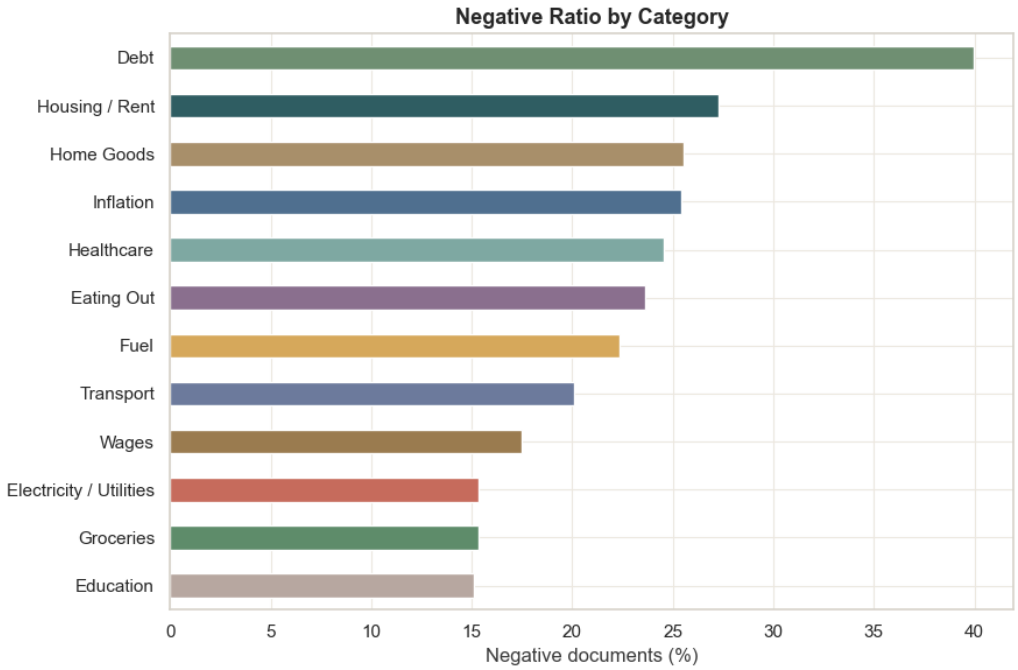

Figure 6: Negative ratio by cost-of-living category

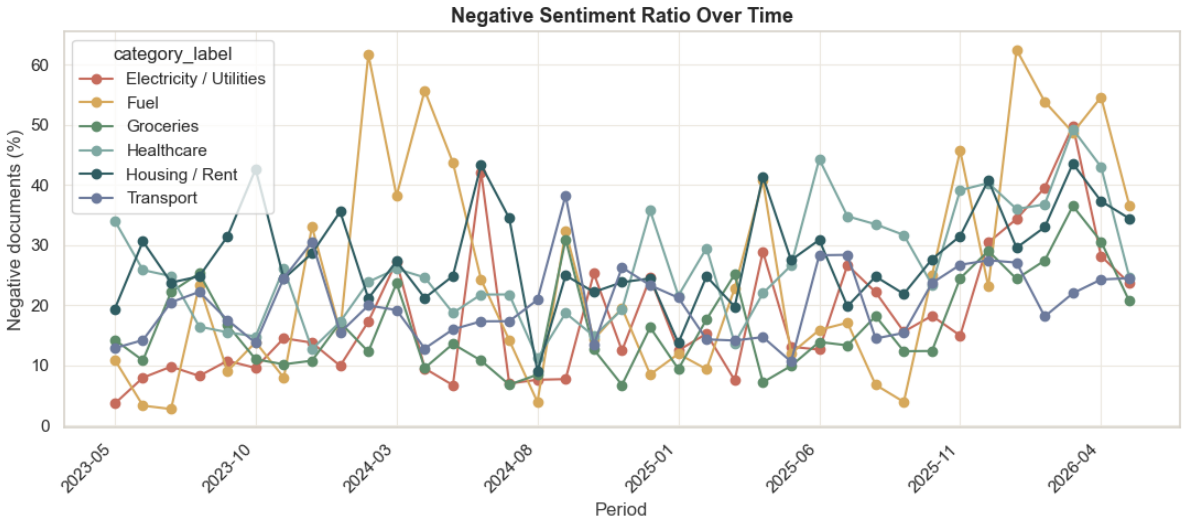

Figure 7: Negative sentiment trend

Negative sentiment is concentrated in categories where household budgets are directly affected, such as housing, groceries, energy, and transport. Trend charts are more useful when combined with source filters because platform volume and media coverage can change independently.

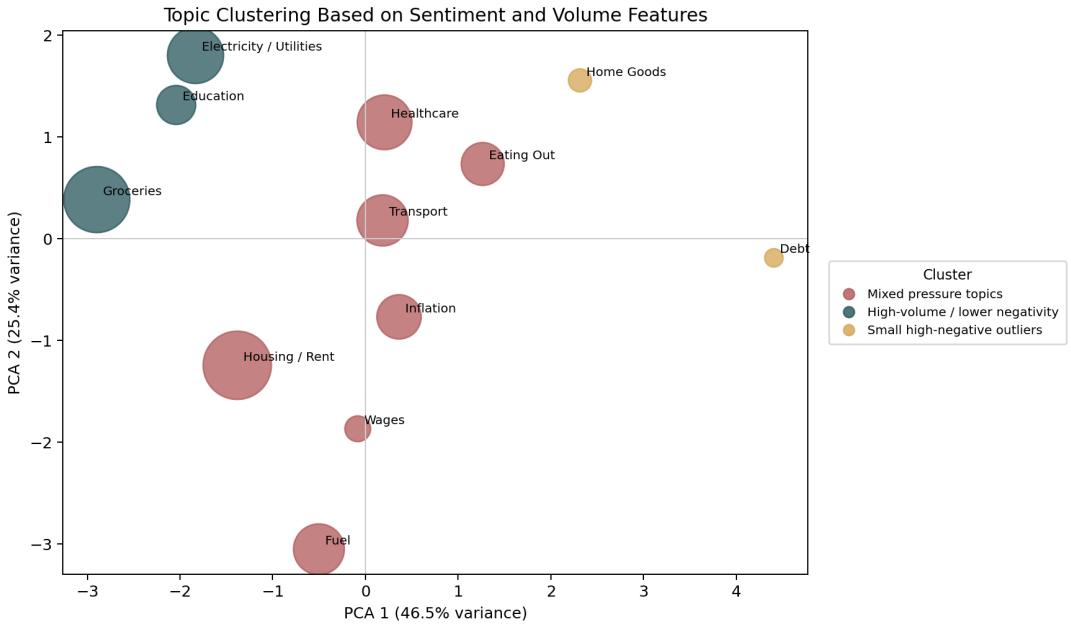

Figure 8: Topic clustering PCA

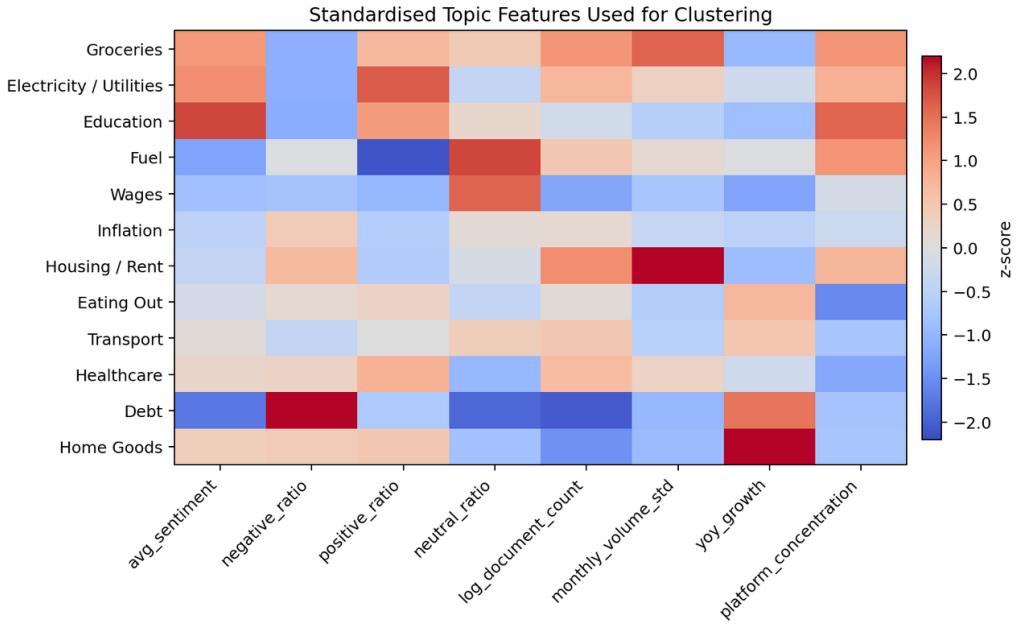

Figure 9: Topic clustering feature heatmap

Topic features show overlap between cost categories. Rent, groceries, fuel, energy, and inflation can appear in the same posts, so rule-based topic labelling should be interpreted as a practical monitoring label rather than a complete semantic model.

7.3 Platform Differences

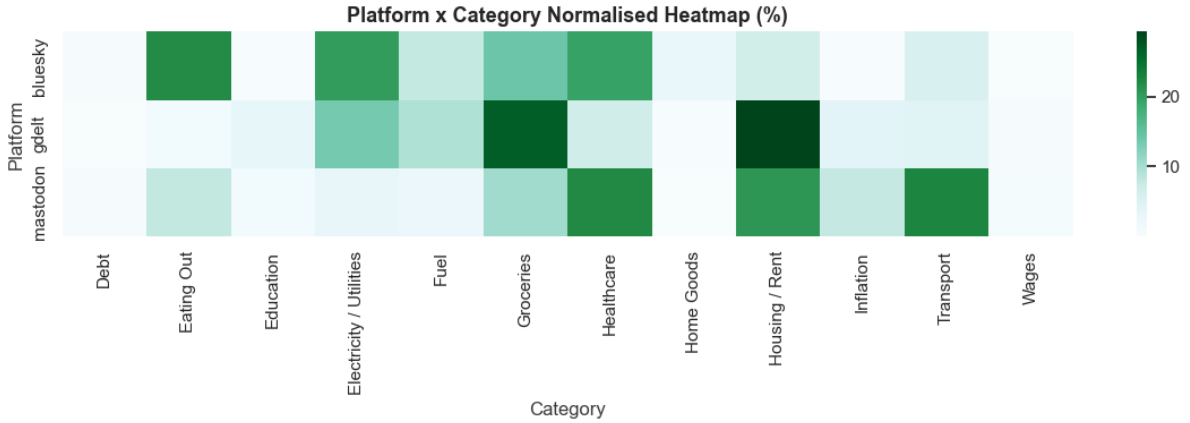

Figure 10: Normalised category distribution by platform

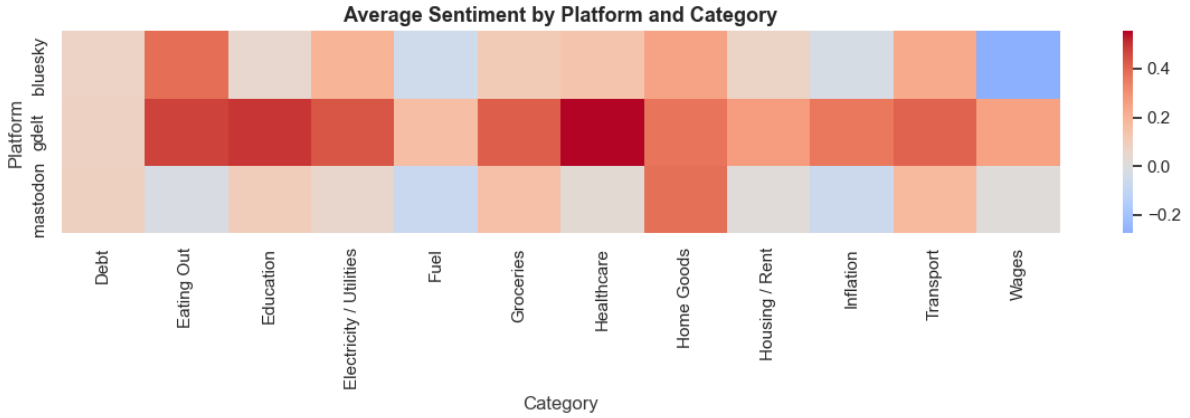

Figure 11: Platform and category sentiment heatmap

Platform-level comparisons are most useful for understanding source bias. Bluesky and Mastodon reflect public social posting behaviour, while GDELT reflects coverage in media records. The API therefore exposes `source_group` and `platform` filters on relevant analytics routes.

7.4 CPI Comparison

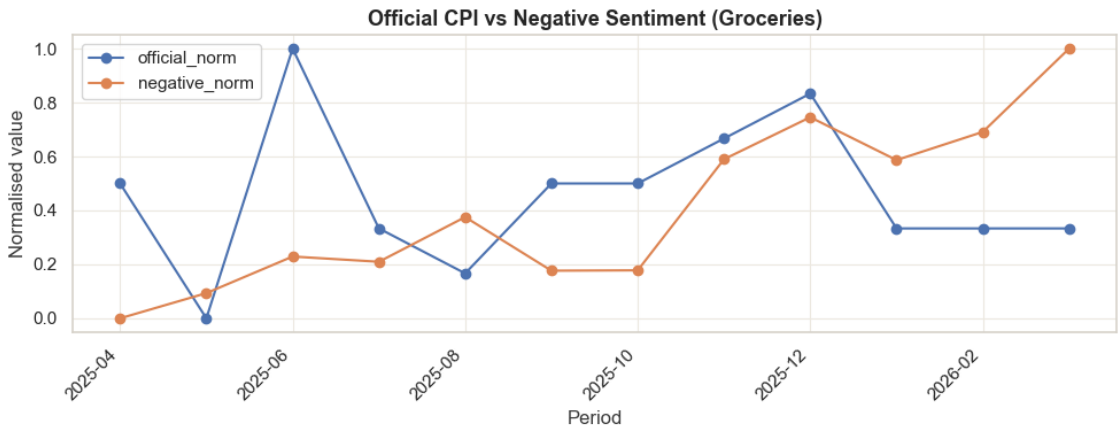

Figure 12: Grocery discussion compared with CPI

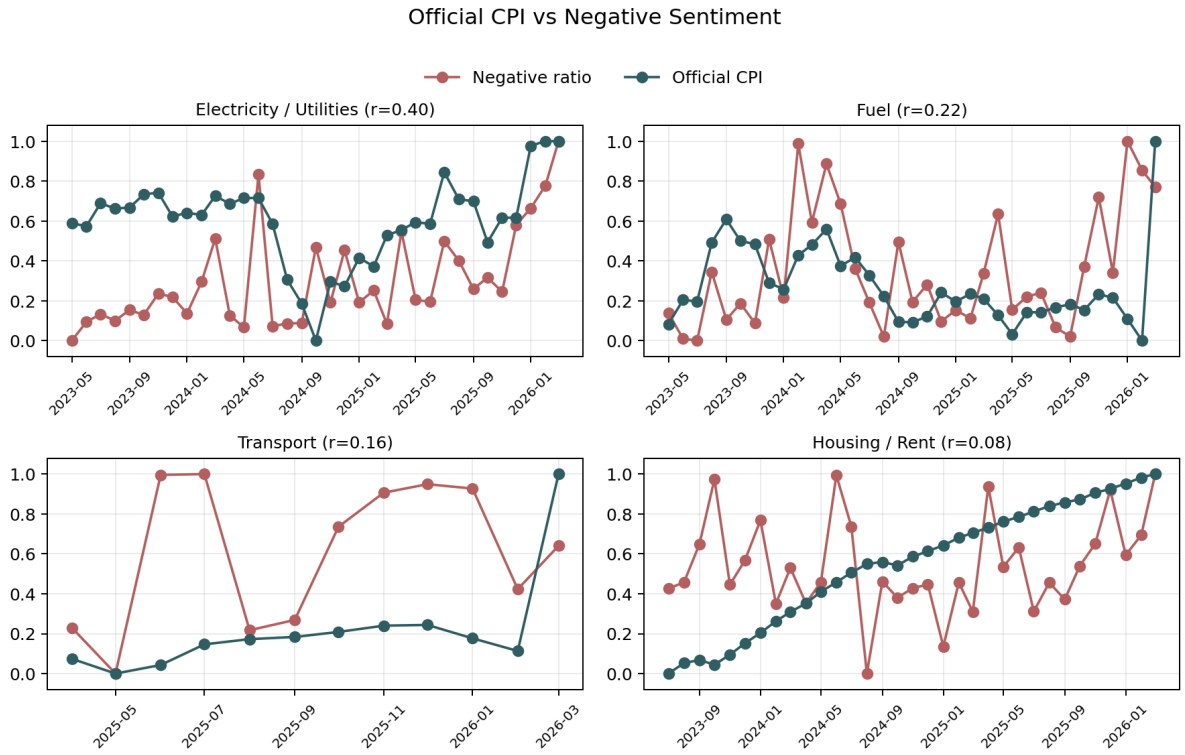

Figure 13: CPI and sentiment comparison grid

CPI comparison gives official context but does not prove causation. CPI is monthly and lagged, while social and media records respond to events, platform behaviour, and collection windows. The comparison is best used as a contextual signal for dashboard exploration.

8 Reliability and Limitations

Table 6: Reliability mechanisms

Area	Risk	Handling
Duplicate records	Overlapping harvest windows or repeated keywords.	Stable source IDs, URLs, document IDs, and Elasticsearch upserts.
External APIs	Rate limits, timeouts, or temporary failures.	Source-specific retries, state checkpoints, and later scheduled recovery.
Raw integration	Different platform schemas.	Unified raw contract with source index and source document identifiers.
NLP processing	Worker crashes or bad messages.	Atomic claiming, stale retry, Redis retry budget, and dead-letter queue.
API usage	Expensive repeated dashboard reads.	Short shared cache TTL, rate limiting, and request-level tracing.
Elasticsearch growth	Processed index expansion.	ILM-managed write alias and stable read alias.

The main limitations remain analytical. Public social platforms are not representative of all Australians. GDELT is media coverage, not public sentiment. Keyword taxonomy is transparent and auditable, but it can miss context and cross-topic meaning. VADER is lightweight and interpretable, but it can misread sar-

casm, slang, political language, quoted text, and metadata-heavy records. City-level analysis is intentionally excluded because source records do not provide reliable city fields.

9 Conclusion

The platform demonstrates a maintainable pattern for cloud-based public text monitoring: source-specific ingestion, unified raw state, queued NLP processing, explicit data contracts, operational API endpoints, and Elasticsearch-backed analytics. The strongest engineering choices are the single API serving path, archive-based GDELT ingestion, Redis-backed runtime diagnostics and worker queueing, KEDA scaling, dead-letter isolation, ILM-managed processed indices, request-level tracing, rate limiting, and a testable FastAPI interface.

Future work should focus on end-to-end container integration tests, API contract tests from the OpenAPI schema, managed dashboard manifests for Prometheus and Grafana, raw-stream lifecycle policies, and a repeatable load-test baseline.

References

[1] Australian Bureau of Statistics. ABS Data API and Consumer Price Index. https://www.abs.gov.au/, 2026. Official CPI data source documentation.

[2] Bluesky. AT Protocol and Bluesky API Documentation. https://docs.bsky.app/, 2026. Public post collection interface documentation.

[3] Elastic. Elasticsearch Documentation. https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html, 2026. Mappings, aliases, aggregations and date histogram documentation.

[4] Fission. Fission Documentation. https://fission.io/docs/, 2026. Function package and timer trigger documentation.

[5] C. J. Hutto and Eric Gilbert. VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. In *Proceedings of the International AAAI Conference on Web and Social Media*, 2014.

[6] Kubernetes. Kubernetes Documentation. https://kubernetes.io/docs/, 2026. Deployment, service, ConfigMap and Secret documentation.

[7] Mastodon. Mastodon API Documentation. https://docs.joinmastodon.org/api/, 2026. Public API documentation.

[8] The GDELT Project. GDELT 2.0 and GKG Documentation. https://www.gdeltproject.org/, 2026. Media data source documentation.